Centralized Processing System (Client–Server Technology)

Introduction

- A **Centralized Processing System** is a computing environment where all major processing, storage and control are handled by a **single powerful computer called server**.
- The **users** interact with this central system (server) through **terminals (clients)**.
- This approach follows the philosophy: **"One powerful brain (server) serving many hands (clients)."**

Example: B**anking systems**, where clerks used terminals connected to a central mainframe that processed all transactions.

- In practice, Centralized Processing is implemented through the **Client–Server Model**.
- In Client – Server Model:
    - **Clients** (user machines/terminals) send **requests** (such as login, transaction and file access).
    - **Server** (central computer) **processes these requests**, performs the necessary computations, fetches data and sends back the **response**.

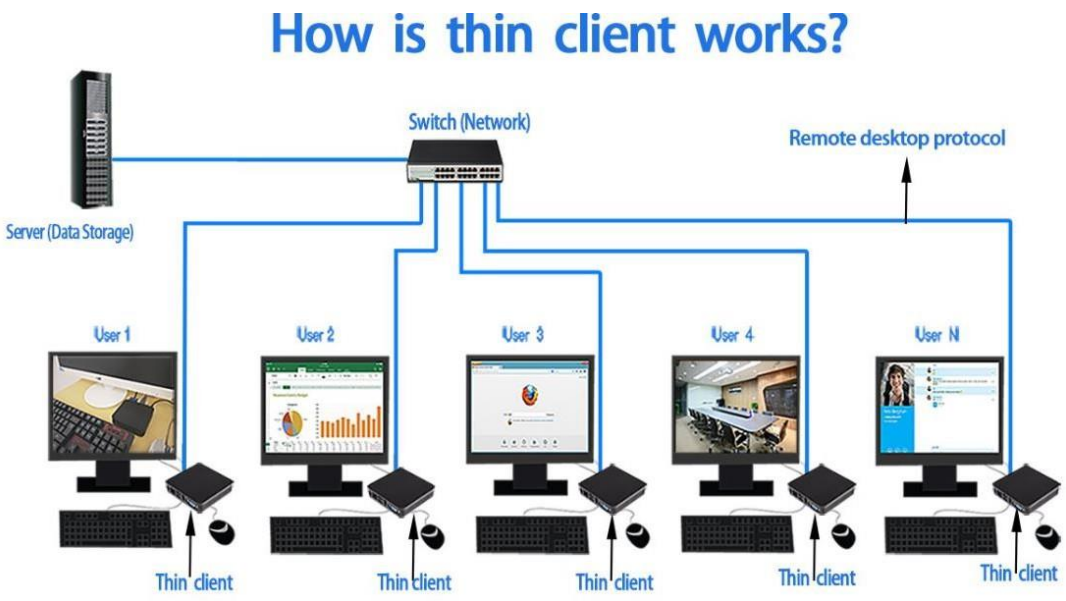
Working of Client–Server System:

1. **Client Request:** The client machine sends a request (ex: Fetch student exam results).
2. **Server Processing:**
    - ✓ Application server executes the business logic.
    - ✓ Database server retrieves required data.
    - ✓ If needed, the file/email/web servers also provide services.
3. **Response Back:** The server sends the processed result back to the client.
4. **Display:** Client machine shows the result to the end-user.

Example: In a railway booking system, the operator enters journey details at a client → request goes to central server → server checks seat availability → response is sent back to client terminal.

Characteristics of Centralized Processing System

1. **Single Processing Unit:** All processing happens at the central server.
2. **Centralized Storage:** Data, applications and programs reside at the server.
3. **Controlled Access:** Users must connect to the server for any operation.
4. **High Dependency:** If the server fails, the whole system stops.
5. **Consistency:** Since all data is centralized, it remains uniform and updated.
6. **Low-cost Terminals:** Clients can be simple machines because the server does the heavy work.



The Server (Central Processing Unit of the System)

The **server** is the backbone of the centralized system. Depending on requirements, different types of servers may be deployed.

Types of Servers

1. **Application Server**
    - ✓ Executes business logic and applications.
    - ✓ Example: Banking server processing account transactions.
2. **Database Server**
    - ✓ Stores, manages, and retrieves structured data.
    - ✓ Example: Oracle or MySQL server storing student records in a university ERP.
3. **File Server**
    - ✓ Provides centralized storage and management of files.
    - ✓ Example: Corporate shared drive for employees' documents.
4. **Web Server**
    - ✓ Hosts and delivers web pages over the internet or intranet.
    - ✓ Example: Apache or Nginx hosting an e-commerce site.
5. **Email Server**
    - ✓ Manages sending, receiving, and storing emails.
    - ✓ Example: Gmail's backend mail servers, Microsoft Exchange Server.

The Client (User Side Terminals)

Clients are the endpoints through which users interact with the centralized system. They can be classified into **two types**:

(a) Thin Client (Dumb Terminal)

- Has little or no processing capability.
- Depends entirely on the server for applications and storage.
- Functions mainly as an **input/output device**.
- **Examples:** Old railway ticket booking terminals, Citrix thin clients, VDI setups.

(b) Thick Client (Smart Terminal)

- Has its own CPU, memory and storage.
- Can run applications locally but still connects to the server for centralized services and data.
- Provides better performance and can work partially offline.
- **Examples:**
  - ✓ Banking system (local processing + server-based banking system).
  - ✓ ATMs (local UI but central banking server for transactions).
  - ✓ Student lab PCs running local software but saving files on a central server.

Comparison: Thin vs Thick Clients

Feature	Thin Client (Dumb Terminal)	Thick Client (Smart Terminal)
Processing Power	None / Very low	Has its own CPU, RAM, storage
Dependency on Server	Complete	Partial (server for data, client for local tasks)
Cost	Cheap	Expensive
Maintenance	Easy (update only at server)	Harder (each client may need updates)
Performance	Slower, depends on server/network	Faster local performance
Examples	Railway terminals, Citrix VDI	PCs, ATMs, lab computers

Advantages of Centralized Processing

- ➢ **Easier Management:** Updates and backups done at one place (server).
- ➢ **Data Consistency:** Centralized database ensures uniformity.
- ➢ **Security Control:** Policies and access managed centrally.
- ➢ **Lower Client Cost:** Thin clients reduce hardware expenses.
- ➢ **Centralized Expertise:** Skilled IT staff can manage a single server.

Disadvantages

- **Single Point of Failure:** If server crashes → all clients stop.
- **Scalability Issues:** Many clients can overload the server.
- **Communication Dependency:** Requires a strong, reliable network.
- **Less User Independence:** Clients cannot operate without server.

Real-Life Examples

- **Banking Systems:** Central server handling all transactions.
- **University ERP:** Student login, results and records from central database.
- **Railway Reservation Systems:** Central server manages all booking data.
- **Email Services:** Clients (Gmail app, Outlook) connecting to email servers.

Decentralized Processing System

Introduction

- ✓ A **Decentralized Processing System (DPS)** is a computing setup where **processing power is distributed across multiple computers (nodes)** instead of being concentrated in a single central server.
- ✓ Each computer (node) has its **own processor, memory and sometimes storage**.
- ✓ Unlike centralized systems, here the philosophy is:
  "Many brains working independently."
- ✓ **Centralized Systems** struggled with **scalability, network dependency and single point of failure**.
- ✓ To overcome this, organizations moved to **decentralized systems**, where multiple smaller computers perform processing tasks independently.
- ✓ Still, decentralized systems can be connected to share selected data but each unit has **autonomy**.

Example: In a multinational company, each branch office has its own computer system to process local data (payroll, accounts and employee attendance) rather than depending only on headquarters.

Characteristics of Decentralized Processing System

1. **Multiple Processing Units:** Processing is distributed across several independent computers.
2. **Autonomy:** Each node can function independently without relying on others.
3. **Local Data Storage:** Each branch/system keeps its own database or files.
4. **Improved Reliability:** Failure of one node does not stop the entire system.
5. **Reduced Network Dependency:** Since processing is local, heavy network traffic is minimized.
6. **Flexibility:** Each unit can use hardware/software suited to its needs.

Advantages of Decentralized Processing

1. **Reliability:** If one branch server fails, others continue working.
2. **Scalability:** Easy to add new branches/nodes without overloading a single system.
3. **Reduced Network Traffic:** Since most requests are handled locally, less data travels over WAN.
4. **Faster Response for Local Tasks:** No need to wait for central server → quicker processing.
5. **Flexibility & Customization:** Each branch/unit can install applications suited to its local needs.

Disadvantages

1. **Data Inconsistency:** Different nodes may store conflicting or outdated data.
2. **Higher Cost:** Multiple servers and IT staff are required at each location.
3. **Complex Management:** Harder to enforce policies across many independent nodes.
4. **Duplication of Effort:** Each branch may repeat similar tasks (e.g., backups, updates).
5. **Integration Challenges:** Sharing and consolidating data across branches may be difficult.

Real-Life Examples

- **Multinational Companies:** Each regional office processes its own accounts and HR data.
- **Universities:** Each department manages its own timetable and student records.
- **Retail Chains (ex: supermarkets):** Each store maintains its own sales and inventory system.
- **Hospitals:** Each hospital branch manages patient data locally, though summaries may be shared with HQ.

Comparison: Centralized vs Decentralized Processing

Feature	Centralized Processing	Decentralized Processing
Processing Location	Single central server	Multiple independent nodes
Dependency	High (server failure stops all)	Low (failure affects only one node)
Data Consistency	High (single database)	Risk of inconsistency
Cost	Cheaper clients, costly central server	Multiple servers → higher overall cost
Management	Easy (centralized IT staff)	Complex (staff needed at each site)
Scalability	Limited (central server bottleneck)	Easy (add more nodes/branches)
Examples	Old banking mainframes, IRCTC (early)	MNC branch offices, supermarkets

3. Distributed Processing

Distributed processing involves multiple interconnected systems that help each other to complete tasks efficiently. In a **distributed system**, a large program is divided into smaller modules and each module runs on different nodes. These nodes communicate with each other and request required modules from other nodes when needed. This means that **every node can act as both a client and a server**, depending on whether it is requesting or providing a service. **Distributed Processing** is ideal for **scalable, high-performance computing**, but managing synchronization can be challenging.

The main features of Distributed Processing

- **Workload Distribution:** Tasks are divided among multiple computing nodes.
- **Network-Dependent:** Systems communicate using high-speed networks.
- **Scalability:** Additional nodes can be added as needed.

Benefits:

- **Fault Tolerance:** If one system goes down, others can continue operations.
- **Faster Processing:** Tasks can be executed in parallel, reducing completion time.
- **Efficient Resource Utilization:** Workloads are spread evenly across available computing power.

Drawbacks:

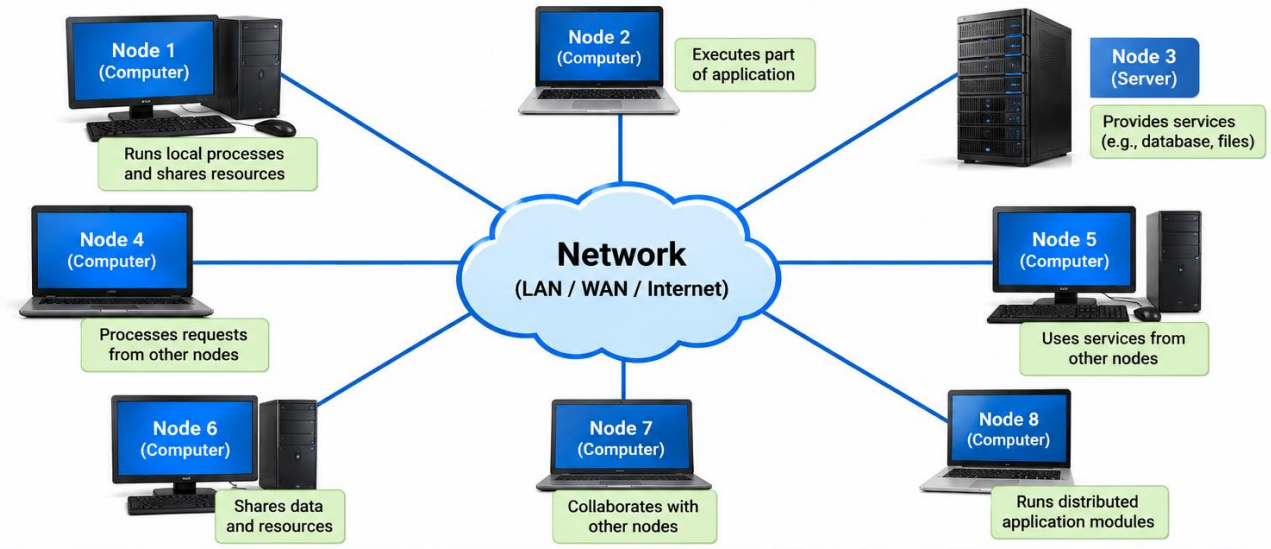
- **Complex System Coordination:** Requires robust communication protocols and data synchronization mechanisms.
- **Security Risks:** More network points mean more potential vulnerabilities.
- **Dependency on Network Reliability:** If the network fails, processing efficiency is affected.

Examples:

- **Cloud Computing Platforms:** Services like Amazon Web Services (AWS) and Microsoft Azure distribute workloads across multiple servers.
- **Blockchain Networks:** Transactions are validated by multiple nodes rather than a single authority.
- **Big Data Processing (Hadoop, Apache Spark):** Data is processed in parallel across multiple servers.

Distributed Computer System

A distributed computer system is a collection of independent computers (nodes) connected through a network and working together to achieve a common goal.



Examples of Distributed Computer Systems

- **Web Applications** (e.g., Google, Facebook)
- **Cloud Computing** (e.g., AWS, Azure)
- **Distributed Databases** (e.g., distributed MySQL, Hadoop)
- **Distributed File Systems** (e.g., HDFS)
- **Cluster Computing** (High Performance Computing)

Management Information System (MIS)

Introduction

A **Management Information System (MIS)** is an organized framework that collects, processes, stores and delivers information to managers for decision-making, planning, control and coordination of business activities.

- Think of MIS as a **bridge between raw data and meaningful decisions**.
- It combines **people, processes and technology** to ensure managers at all levels get the right information at the right time.
- MIS is not just about technology—it is about **using IT systems** (like centralized or decentralized processing) to **support business goals**.
- In a **Centralized Processing System (CPS)**, data from client machines flows to a powerful central server. MIS can be built on top of this to provide consolidated reports, dashboards and analysis.

  Example: a **bank's head office server** processes all branch transactions and produces daily MIS reports for top management.
- In a **Decentralized Processing System (DPS)**, multiple nodes or local servers exist across branches. MIS here collects data from all nodes, integrates it and gives both **local managers** (branch-specific MIS) and **top managers** (organization-wide MIS) the required information.

  Example: a **retail chain** where each store has its own server for sales and MIS aggregates them into company-level sales analytics.

Working of MIS (How It Processes Requests)

MIS generally works in four stages, similar to client-server request processing:

1. **Data Collection**
   - ✓ From different sources: transactions, databases, decentralized branches, or centralized servers.

     Example: Customer purchases from POS machines, employee attendance logs, or supply chain data.
2. **Data Processing**
   - ✓ Data is cleaned, validated, summarized, and converted into meaningful form.

     Example: Daily sales transactions processed into total revenue, per-product sales, or per-region sales.
3. **Information Storage**
   - ✓ Processed data is stored in databases (centralized or distributed) for future reference.

     Example: A central Data Warehouse in CPS or multiple local storage points in DPS.
4. **Information Dissemination**
   - ✓ Reports, dashboards, alerts and visualizations are delivered to managers via client systems.

     Example: A sales manager sees **weekly sales trends** on a dashboard; the CEO gets **company-wide MIS reports**.

Characteristics of MIS

A good MIS has the following features:

1. **Management-Oriented** – Designed to serve managers at operational, tactical and strategic levels.
2. **Integrated** – Combines information from multiple functional areas (sales, HR, finance, production).
3. **Timely** – Provides information at the right time for quick decision-making.
4. **Accurate & Reliable** – Eliminates duplication and ensures data correctness.
5. **User-Friendly** – Easy to access via dashboards, reports, or alerts.
6. **Supports Decision-Making** – Gives summarized data and analytical insights rather than raw transactions.
7. **Flexible** – Can adapt to changing organizational needs and new technologies.

Types of MIS

MIS can take multiple forms depending on the purpose:

1. **Transaction Processing Systems (TPS)**
   - ✓ Focus on daily routine operations.

     Example: ATM transactions, online ticket booking.
2. **Decision Support Systems (DSS)**
   - ✓ Helps managers analyse "what-if" scenarios.

     Example: Deciding whether to increase product price by 5% and analyzing projected sales impact.
3. **Executive Information Systems (EIS)**
   - ✓ Designed for senior executives with visual dashboards and KPIs.

     Example: A CEO's dashboard showing revenue growth, customer churn, and profits.
4. **Knowledge Management Systems (KMS)**
   - ✓ Stores organizational knowledge for innovation and learning.
   - ✓ Example: Internal wiki for policies, project documentation, or lessons learned.

Examples of MIS in Action

- **Banking**: MIS generates daily branch-wise reports of deposits, withdrawals, and loan disbursements.
- **Healthcare**: MIS provides doctors with patient history, hospital resource usage, and billing insights.
- **Retail**: MIS helps track sales per product category, inventory turnover, and seasonal demand patterns.
- **Education**: MIS allows universities to manage student records, exam results, and faculty performance.